

MERN Application Deployment Report

Infrastructure Automation with Terraform & Ansible

1. Executive Summary & Objective

The objective of this project was to automate the end-to-end infrastructure provisioning and application configuration for the **TravelMemory** MERN-stack application on AWS.

Using **Terraform**, an isolated multi-subnet Virtual Private Cloud (VPC) was created along with necessary security groups, routing rules, and EC2 instances. Using **Ansible**, configuration management was automated to setup Node.js, PM2, and Nginx on the public web server, and secure MongoDB on the isolated database server.

- **GitHub Repository:** <https://github.com/vivek-rajendran/TravelMemory>
- **Application URL:** <http://65.0.3.141>

2. Infrastructure Architecture Overview (Terraform)

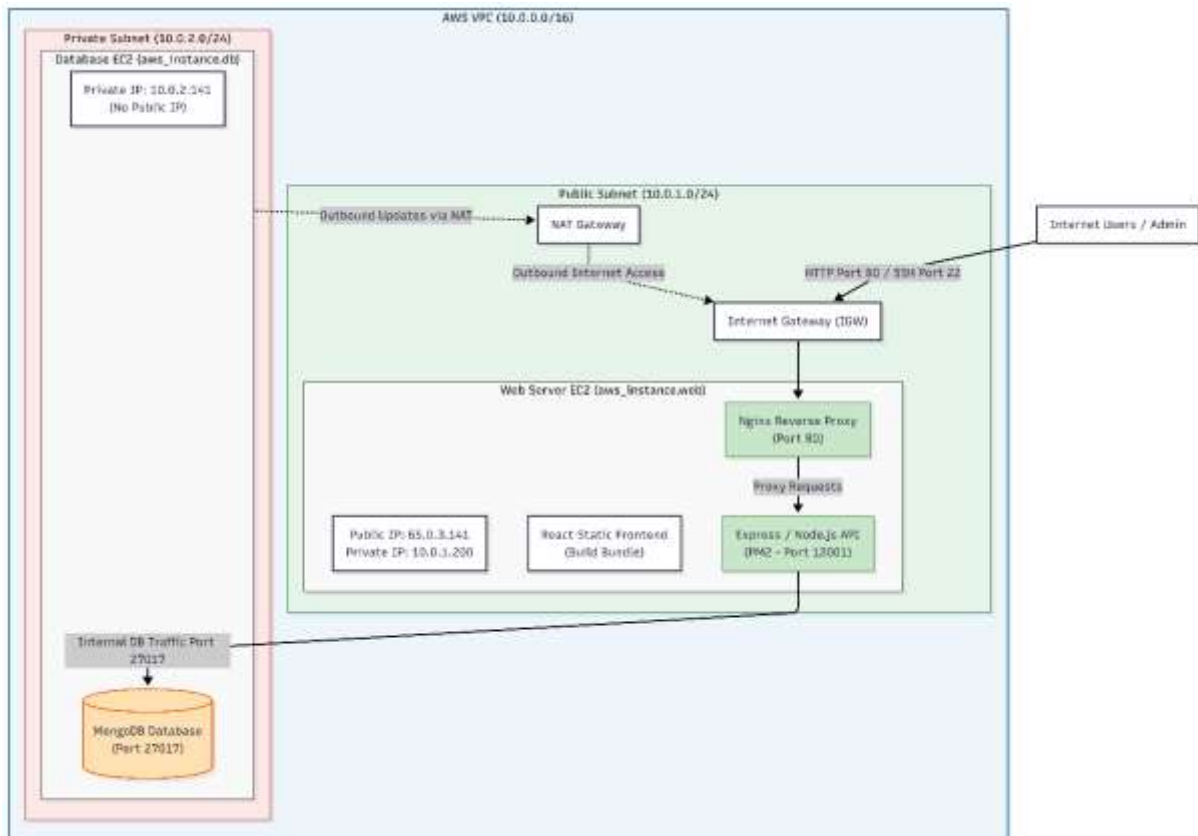
The underlying cloud infrastructure was declared and provisioned using Terraform scripts (**main.tf**, **variables.tf**, **outputs.tf**).

2.1 Terraform Implementation & Execution

The AWS infrastructure was provisioned using Terraform. The following commands were used during the infrastructure provisioning process:

1. terraform init – Initializes the Terraform working directory and downloads the required providers.
2. terraform validate – Validates the Terraform configuration files.
3. terraform plan – Creates an execution plan showing the resources that Terraform will provision.
4. terraform apply – Provisions the AWS infrastructure including the VPC, subnets, gateways, route tables, security groups, and EC2 instances.
5. terraform output – Displays the Terraform output values, including the public IP address of the web server.

2.2 Network & Compute Specifications



- **VPC Network:** 10.0.0.0/16 CIDR block providing network boundary isolation.
- **Public Subnet (10.0.1.0/24):** Associated with an Internet Gateway (IGW) to handle inbound/outbound traffic for the public web node.
- **Private Subnet (10.0.2.0/24):** Associated with a NAT Gateway located in the public subnet, allowing the database instance to download software updates securely without exposing public IP addresses.
- **Web Server EC2 (aws_instance.web):** Provisioned in the Public Subnet with Elastic/Public IP 65.0.3.141 and private IP 10.0.1.200.

Database EC2 (aws_instance.db): Provisioned in the Private Subnet with internal IP 10.0.2.141 and no public IP assigned.

3. Configuration Management & Deployment (Ansible)

Configuration and deployment were divided into modular Ansible playbooks executed against an inventory defined in **inventory.ini**.

3.1 Ansible Execution Commands

The following Ansible commands were used to verify connectivity and execute the configuration and deployment playbooks:

```
ansible all -m ping
```

ansible-playbook setup-web.yml

ansible-playbook setup-db.yml

ansible-playbook deploy-app.yml

Bastion Host / ProxyJump Pattern

To communicate with the database instance isolated inside the private subnet, Ansible was configured using SSH ProxyJump through the web server:

Ini, TOML

[db:vars]

```
ansible_ssh_common_args='-o StrictHostKeyChecking=no -o
UserKnownHostsFile=/dev/null -o ProxyJump=ubuntu@65.0.3.141'
```

Playbook Breakdown

1. Web Server Setup (setup-web.yml):

- Installed Node.js (v18+), NPM, PM2 process manager, and Nginx web server.
- Configured initial system permissions and dependencies.

2. Database Setup (setup-db.yml):

- Automated installation of MongoDB Community Edition on 10.0.2.141.
- Updated mongod.conf to bind MongoDB strictly to local network interface 0.0.0.0 or internal private IP 10.0.2.141, enabling connections from the web subnet while blocking public access.

3. Application Deployment (deploy-app.yml):

- **Backend:** Cloned TravelMemory repo, installed dependencies, configured .env with PORT=12001 and MONGO_URI=mongodb://10.0.2.141:27017/travelmemory, and started the process under PM2.
- **Frontend:** Injected REACT_APP_BACKEND_URL="http://65.0.3.141" during npm run build compilation to ensure clients send API calls to the public IP endpoint.
- **Nginx Reverse Proxy:** Configured Nginx to serve the static frontend React build on port 80 and reverse-proxy API requests matching /trip and /url to http://127.0.0.1:12001.

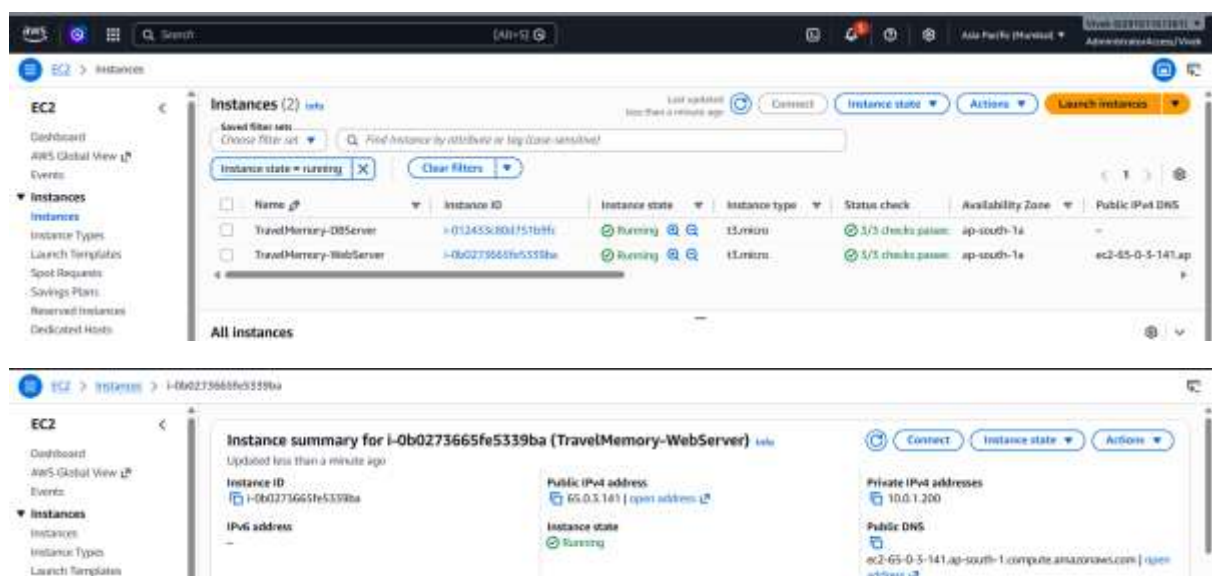
4. Security Hardening & Isolation

- **Network Perimeter Hardening (db_sg):** The database Security Group ingress rules restrict SSH (Port 22) and MongoDB (Port 27017) traffic **exclusively** to the Web Subnet CIDR block (10.0.1.0/24). direct access from the internet is completely blocked.
- **Web Security Group (web_sg):** Allows public ingress on Port 80 (HTTP) for application users, while SSH access on Port 22 is restricted to authorized administrative IPs.
- **SSH Key Pair Authentication:** Password-based SSH authentication was disabled across all nodes in favor of asymmetric SSH key pairs (travelmemory-key).
- **Non-Root Execution:** Node.js services are run under a dedicated system user (ubuntu) managed via PM2 process manager rather than system root.

5. Verification & Deliverable Evidence

1. [SCREENSHOT 1]: AWS Console

- **Screenshot 1: AWS Infrastructure – Web and Database EC2 Instances**
Terraform-provisioned web and database EC2 instances running in the public and private subnets respectively.
- **Description:** Showing web_server (Public IP 65.0.3.141) and db_server (Private IP 10.0.2.141) in running state.





2. [SCREENSHOT 2]: Ansible Execution

○ Screenshot 2: Ansible Connectivity Verification

Ansible successfully connects to both the public web server and private database server through the configured SSH ProxyJump.

- **Description:** Output of ansible all -m ping returning "ping": "pong" across both hosts via ProxyJump.

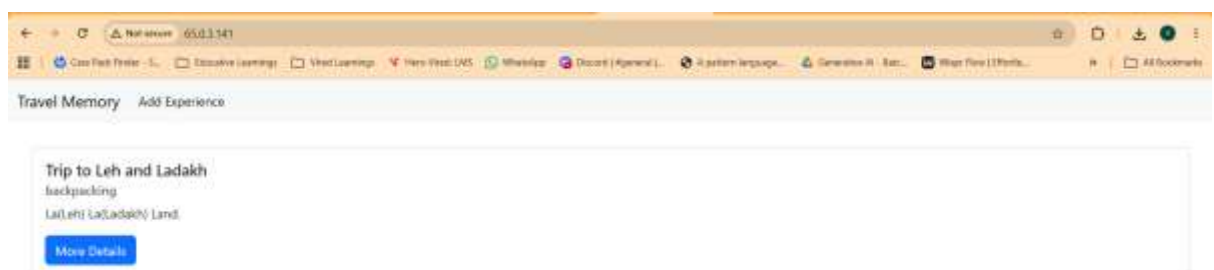
```
vikra@LAPTOP-46PSCUTR:~/projects/travelmemory/ansible$ cd ~/projects/travelmemory/ansible
vikra@LAPTOP-46PSCUTR:~/projects/travelmemory/ansible$ eval $(ssh-agent -s)
Agent pid 86377
vikra@LAPTOP-46PSCUTR:~/projects/travelmemory/ansible$ ssh-add ~/.ssh/travelmemory-key
Identity added: /home/vikra/.ssh/travelmemory-key (vikra@LAPTOP-46PSCUTR)
vikra@LAPTOP-46PSCUTR:~/projects/travelmemory/ansible$ ansible all -m ping
web_server | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.10"
  },
  "changed": false,
  "ping": "pong"
}
db_server | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.10"
  },
  "changed": false,
  "ping": "pong"
}
```

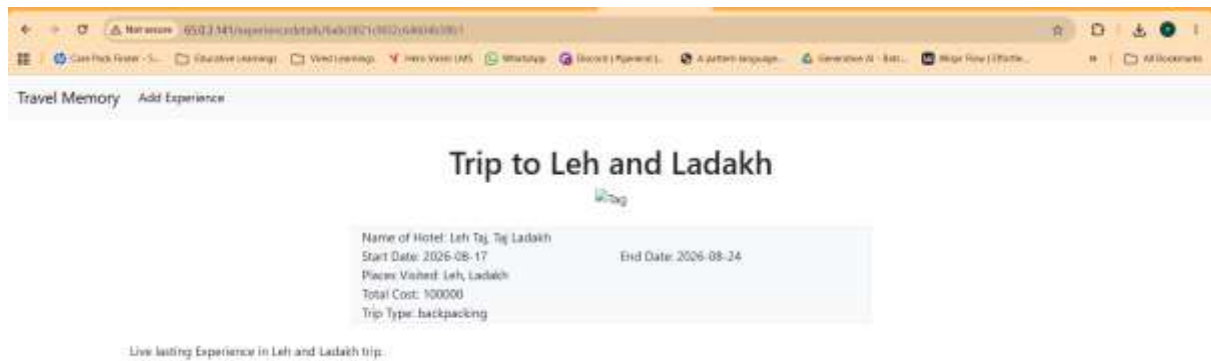
3. [SCREENSHOT 3]: Live Application Interface

○ Screenshot 3: TravelMemory Application – Successful Deployment

The TravelMemory MERN application is successfully deployed and accessible through the public web server.

- **Description:** TravelMemory application rendered in browser at http://65.0.3.141 (<http://65.0.3.141>).





4. [SCREENSHOT 4]: Trip API Verification

- **Screenshot 4: Trip API Verification – Successful POST Request**

Successful POST /trip request returning HTTP 200 OK, confirming communication between the React frontend, Node.js backend, and MongoDB database.

- **Description:** API CURL showing successful 200 OK POST response for /trip endpoint calls.

```
vikra@LAPTOP-46P5CUTH:~/projects/travelmemory/ansible$ curl -i -X POST http://65.0.3.141/trip \
-H "Content-Type: application/json" \
-d '{"tripName":"Coimbatore Outing", "nightStay":2, "tripBudget":5000, "message":"Awesome deployment!"}'
HTTP/1.1 200 OK
Server: nginx/1.18.0 (Ubuntu)
Date: Mon, 24 Aug 2026 09:29:55 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 20
Connection: keep-alive
X-Powered-By: Express
Access-Control-Allow-Origin: *
ETag: W/"14-vavtkRsflrUY8Yc+q4ufr7ggNI"
```

```
SOMETHING WENT WRONGvikra@LAPTOP-46P5CUTH:~/projects/travelmemory/ansible$ curl -i -X GET http://65.0.3.141/trip
HTTP/1.1 200 OK
Server: nginx/1.18.0 (Ubuntu)
Date: Mon, 24 Aug 2026 09:30:06 GMT
Content-Type: application/json; charset=utf-8
Content-Length: 2613
Connection: keep-alive
X-Powered-By: Express
Access-Control-Allow-Origin: *
ETag: W/"a35-jTV6dAxx7ABv8FKLLRubzK4awCU"
```

```
[{"_id":"6a0c8221c432c64684b58b1","tripName":"Trip to Leh and Ladakh","startDateOfJourney":"2026-08-17","endDateOfJourney":"2026-08-24","nameOfHotels":"Leh Taj, Taj Ladakh","placesVisited":"Leh, Ladakh","totalCost":100000,"tripType":"backpacking","experience":"Live lasting Experience in Leh and Ladakh trip","image":"","shortDescription":"La(Leh) La(Ladakh) Land","featured":false,"createdAt":"2026-08-24T08:57:56.502Z","_v":0},{"_id":"6a8c8ed4c432c64684b58b7","tripName":"Athirapalli Waterfalls","startDateOfJourney":"2026-08-24","endDateOfJourney":"2026-08-25","nameOfHotels":"Merala's Niagara","placesVisited":"Athirapalli Falls","totalCost":10000,"tripType":"backpacking","experience":"Athirapalli Experience - Short Trip for rejuvenate","image":"","shortDescription":"Athirapalli Experience","featured":false,"createdAt":"2026-08-24T08:57:56.502Z","_v":0},{"_id":"6a8c8ed4c432c64684b58b9","tripName":"Athirapalli Waterfalls","startDateOfJourney":"2026-08-24","endDateOfJourney":"2026-08-25","nameOfHotels":"Merala's Niagara","placesVisited":"Athirapalli Falls","totalCost":10000,"tripType":"backpacking","experience":"Athirapalli Experience - Short Trip for rejuvenate","image":"","shortDescription":"Athirapalli Experience","featured":false,"createdAt":"2026-08-24T08:57:56.502Z","_v":0},{"_id":"6a8c8ed4c432c64684b58bb","tripName":"Athirapalli Waterfalls","startDateOfJourney":"2026-08-24","endDateOfJourney":"2026-08-25","nameOfHotels":"Merala's Niagara","placesVisited":"Athirapalli Falls","totalCost":10000,"tripType":"backpacking","experience":"Athirapalli Experience - Short Trip for rejuvenate","image":"","shortDescription":"Athirapalli Experience","featured":false,"createdAt":"2026-08-24T08:57:56.502Z","_v":0}]]vikra@LAPTOP-46P5CUTH:~/projects/travelmemory/ansible$
```